

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Scuola di Scienze Matematiche
Fisiche e Naturali

Corso di Laurea Triennale
in Matematica

Graph Neural Networks

At the core of a Reinforcement Learning framework for Graph Theory conjectures

Candidate:
Giovanni Ufrasi

Supervisor:
Prof. Maurizio Parton

Co-supervisor:
Prof. Andrea Frosini

Academic Year 2025/2026

1 Introduction

Breakthroughs such as the Four Color Theorem in 1976 by Appel and Haken [1], and the proof of the Kepler conjecture in 1998 by Hales [2] are remarkable examples of computer-assisted proofs. In these cases, algorithms were explicitly programmed by mathematicians to perform exhaustive checking of finite yet massive sets of cases. Recently, however, machine learning methods have demonstrated extraordinary performance in environments with enormous state spaces, such as Atari games [3] and Go [4], through autonomous learning from interaction and, in the latter case, through self-play. In his work [5] Adam Z. Wagner proposed the application of reinforcement learning algorithms to find explicit counterexamples to conjectures in combinatorics and graph theory.

1.1 Wagner's Framework

In graph theory, conjectures often state that a property holds for every graph in a given class, or that an invariant satisfies a certain bound. For graphs with a fixed number n of vertices, the search for a counterexample is a finite problem, since only finitely many non-isomorphic graphs of order n exist. However, the size of this search space grows super-exponentially with n, quickly making exhaustive verification infeasible. Wagner's framework [5] replaces exhaustive search with a targeted optimization procedure.

Many graph-theoretic conjectures can be formulated as $f(G) < 0$ or $f(G) \leq 0$, for every graph G in a class $\mathcal{G}$, where f is a suitable evaluation function. For instance, a conjecture now disproved by Wagner [5] stated that $\lambda_1(G) + \mu(G) \geq \sqrt{n-1} + 1$ for every connected graph G with $n \geq 3$ vertices, where $\lambda_1(G)$ is the largest eigenvalue of its adjacency matrix and $\mu(G)$ is its matching number. Setting $f(G) = \sqrt{n-1}+1-\lambda_1(G)-\mu(G)$, the conjecture becomes $f(G) \leq 0$, and a counterexample is a graph with $f(G) > 0$. Similarly, a conjecture of the form $f(G) < 0$ is disproved by a graph with $f(G) \geq 0$.

Wagner frames this objective as a maximization problem through Reinforcement Learning (RL). Intuitively, RL is a framework in which an agent learns, through repeated interaction with an environment, how to play a game so as to maximize a numerical score (see Section 3). In this sense, RL can be thought of as a derivative-free optimization method; see, for instance, [6]. In our case, the game consists of iteratively modifying edges among a fixed set of vertices to build a graph with a large value of f. Figure 1 shows the evolution of the best graph during training for the conjecture mentioned above.

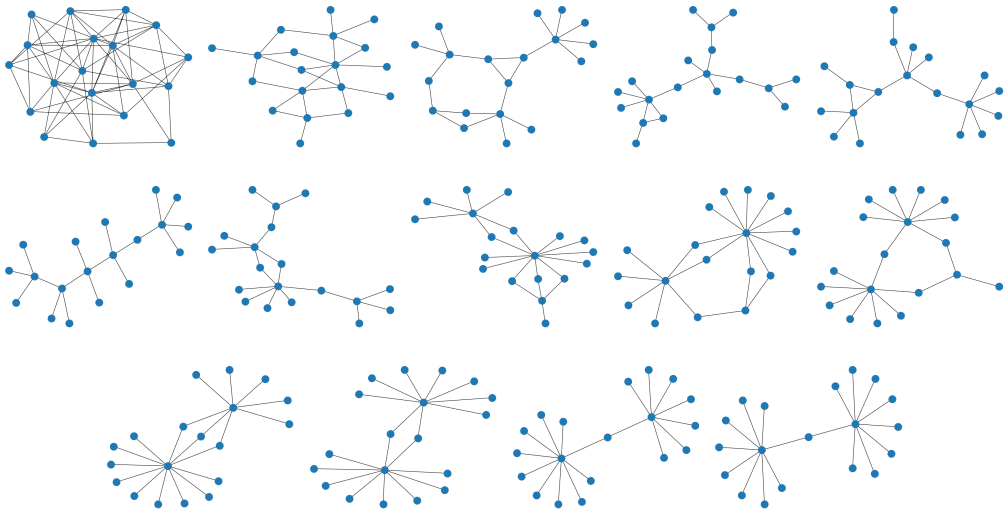


Figure 1: Evolution of the best graph during training for the conjecture on $\lambda_1(G) + \mu(G)$, of which the final result is a counterexample. Reproduced from [5, Fig. 3].

We will informally refer to this family of graph-construction environments as the "build-your-graph" game, independently of whether the allowed moves add, delete, or modify edges. Figure 2 illustrates Wagner's original sequential variant. The agent iteratively constructs a graph, typically using a neural network called the *policy* to choose its moves. The feedback received by the agent is called the *reward*: it may be the final score $f(G_T)$, or an incremental reward $f(G_{t+1}) - f(G_t)$ called *telescoping*, since its sum over an episode is $f(G_T) - f(G_0)$. Over many training episodes, the agent uses these rewards to improve its policy and generate graphs with increasingly large scores.

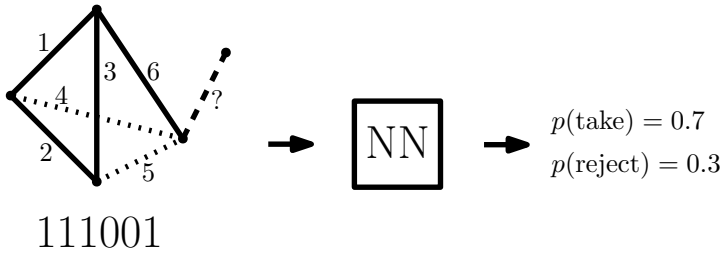


Figure 2: Wagner's sequential build-your-graph game, with a fixed, arbitrarily chosen ordering of possible edges. The neural network (NN) assigns probabilities to including or excluding the next edge. The partial string `111001` encodes the inclusion of edges $1, 2, 3, 6$ and the exclusion of edges $4, 5$ (dotted). Reproduced from [5, Fig. 1].

If the agent finds a graph that violates the conjectured inequality, a counterexample has been obtained. Failure to find one does not prove the conjecture: a counterexample may not exist in the tested range, may be very rare, or may simply be missed by the optimization procedure. Nevertheless, the experiment can provide empirical evidence and reveal structural features of high-scoring graphs. Explainability techniques may also help identify such features, as in [7].

1.2 Neural Architecture Selection

As described above, the agent's moves are decided by a neural network which evaluates the current graph. Representing a graph as input, however, raises an important issue: two isomorphic graphs may have different adjacency matrices, and the output of a network that is not permutation invariant may depend on the chosen vertex labelling (Section 2). Graph Neural Networks (GNNs) overcome this representation issue by being equivariant at node level and invariant at graph level (Section 5). Most commonly used GNNs are based on *message passing*; two of them, GCN and GIN, will be discussed in detail later.

Among the many existing models, the choice of the GNN architecture when applying Wagner's framework to a graph-theoretic conjecture can be delicate. Testing different models and hyperparameters before embedding them into the RL agent may help identify an architecture better suited to the graph invariants involved in the conjecture.

1.3 Related Work

Wagner introduced his approach in [5]. Mehrabian et al. later introduced a more flexible edge-flipping environment and compared an AlphaZero-based approach with tabu search, obtaining new lower bounds for an extremal graph problem of Erdős [8].

Several subsequent works applied or extended Wagner's approach. To cite a few, Stevanović et al. considered a conjecture involving graph energy and matching number; Ghebleh et al. developed a faster reimplementation and applied it to Laplacian spectral-radius conjectures, small Ramsey numbers, and graphs with two main eigenvalues; Bouffard and

Breen proposed a parallelized implementation; and Damnjanović et al. recently introduced the general RLGT framework for extremal graph theory [9–14].

Finally, Angileri et al., from the CuriosAI research group, systematically investigated Wagner's framework by introducing several build-your-graph environments and a graph dataset containing a wide range of structural and spectral features. They later analyzed different environment dynamics, RL algorithms, neural architectures, and reward schemes, using Brouwer's conjecture as a case study [15, 16].

The next chapters introduce the graph theory, RL, and neural network notions needed for this work, followed by a focus on the graph isomorphism problem, GNNs and how to measure their representational power. The final section presents the experimental contribution of this thesis, which grew out of an ongoing collaboration with the CuriosAI research group while working on Brouwer's conjecture, which has since been proven [17]. The experiments compare GCN and GIN on a supervised learning task using the group's graph dataset, included as supplementary material to this thesis. The results are discussed in light of a formal justification of the difference in expressive power between the two models, which may explain GIN's better performance.

2 Graph Theory

Definition (Graph)

A *graph* is a pair $G = (V, E)$, where V is a finite set whose elements are called *vertices* or *nodes*, and E is a set of unordered pairs $\{v_1, v_2\}$ of distinct vertices, whose elements are called *edges*.
The *order* of a graph is the number of its vertices, denoted by $|V|$; the *degree* of a vertex is the number of edges incident to it.

The definition provided above refers to *simple undirected graphs*, meaning that the edges are not oriented and there are no self-loops or multiple edges. For convenience, we assume that for any graph G of order n, the vertex set is $V(G) = \{0, 1, 2, \ldots, n-1\}$.

Given a graph G, the set $E(G)$ defines a symmetric relation on the vertices called the *adjacency relation*: two vertices u and v are said to be adjacent if $\{u, v\} \in E(G)$. Once the vertices are labelled, a graph is fully determined by an $n \times n$ *adjacency matrix*

$$A(G)_{ij} = \begin{cases} 1 & \text{if } \{i, j\} \in E(G), \\ 0 & \text{otherwise.} \end{cases}$$

For simple graphs, $A(G)_{ii} = 0$ for all i. As we consider only undirected graphs, the adjacency matrix is symmetric ($A_{ij} = A_{ji}$), so all of its eigenvalues are real. Those will also be referred to as the *eigenvalues* of G: relabelling the vertices produces a similar adjacency matrix and therefore preserves its eigenvalues.

The *degree matrix* $D(G)$ is the $n \times n$ diagonal matrix with $D(G)_{ii} = \deg(i)$ for every vertex $i \in V(G)$.The *Laplacian matrix* $L(G)$ is the $n \times n$ matrix $L(G) = D(G) - A(G)$.

Definition (Isomorphism)

An *isomorphism* between two graphs G and H is a bijection $f\colon V(G) \to V(H)$ such that any two vertices u and v of G are adjacent in G if and only if $f(u)$ and $f(v)$ are adjacent in H. If there exists an isomorphism between G and H, the graphs are said to be *isomorphic*, and denoted by $G \simeq H$. Equivalently, $G \simeq H$ if and only if $A(H) = PA(G)P^T$ for some permutation matrix P.

Whether graph isomorphism can be decided in polynomial time remains an open problem. We will discuss this issue later in Section 5.2. To introduce heuristic methods based on color-refinement for this problem, it is useful to define vertex coloring:

Definition (Vertex Coloring)

Given a set $\mathcal{C}$ of labels called *colors*, a *vertex coloring* (or simply *coloring*) of a graph G is a mapping $c\colon V(G) \to \mathcal{C}$. A coloring using at most k colors is called a *k-coloring*.

A coloring is called *proper* if no two adjacent vertices share the same color. We do not require this condition here.

To study graph features independently of a particular vertex labelling, we introduce the following definitions. By a slight abuse of notation, we write $f(A)$ and $F(A)$ for the functions evaluated on the graph represented by A.

Definition (Invariance)

Let $\mathcal{G}$ be the class of all graphs of order n, and let X be an arbitrary set. A graph-level function $f\colon \mathcal{G} \to X$ is *permutation invariant* if it maps isomorphic graphs to the same element. Equivalently, for any adjacency matrix A and any permutation matrix P,

$$f(PAP^T) = f(A).$$

Definition (Equivariance)

Let $\mathcal{G}$ be the class of all graphs of order n, and let X be an arbitrary set. A node-level function $F\colon \mathcal{G} \to X^n$ is *permutation equivariant* if, for any adjacency matrix A and permutation matrix P,

$$F(PAP^T) = PF(A),$$

Functions that are permutation invariant are simply called *graph invariants*. Some key examples include:

- **Order:** the number of vertices;
- **Diameter:** for a connected graph, the maximum shortest-path distance between any pair of vertices;
- **Matching number:** the maximum size of a set of edges with no common endpoints;
- **Clique number:** the number of vertices in a maximum clique (that is, a complete subgraph);
- **Independence number:** the number of vertices in a maximum independent set (that, is a set of pairwise non-adjacent vertices);
- **Adjacency spectral radius:** $\rho(A(G))$, the maximum absolute value of an eigenvalue of the adjacency matrix;
- **Laplacian spectral radius:** $\rho(L(G))$, the maximum absolute value of an eigenvalue of the Laplacian matrix.

These invariants were chosen as illustrative examples because their values are provided for every graph in the CuriosAI dataset, described in Section 6.1.

The graph structure alone encodes the connectivity among its vertices, but in applications each vertex $i \in V(G)$ may also carry local information represented by a feature vector.

Definition (Node Features)

Let $G = (V, E)$ be a graph of order n. A *node feature vector* is a vector $h_i \in \mathbb{R}^d$ associated with each vertex $i \in V(G)$, where d is the number of node features. The corresponding *node feature matrix* is a matrix $H \in \mathbb{R}^{n \times d}$ whose rows are the transposed feature vectors h_i^T, ordered according to the vertex labelling.

When node features are present, a relabelling represented by a permutation matrix P transforms the pair (A, H) into (PAP^T, PH). Accordingly, permutation invariance and equivariance require, respectively,

$$f(PAP^T, PH) = f(A, H) \qquad \text{and} \qquad F(PAP^T, PH) = PF(A, H).$$

The node feature matrix is typically given as input to a graph neural network,together with the graph structure, represented either by an edge list or an adjacency matrix. For a graph with no additional node information, dummy features, such as $h_i = 1$ for every vertex i, can be used.

3 Reinforcement Learning

Alongside supervised and unsupervised learning, *Reinforcement Learning* (RL) is one of the main machine learning paradigms. It studies how an intelligent agent can learn how to act through interaction with an environment in order to maximize the expected return. Following the standard literature [18], this interaction is formally modeled as a *Markov decision process*. Since the environments considered in this thesis are finite, we restrict our attention to *finite* Markov decision processes.

Definition (Markov Decision Process)

A *finite Markov Decision Process* (MDP) is a 4-tuple $\mathcal{M} = (S, A, \mathcal{R}, p)$, where:

- S is a finite set called the **state space**;
- A is a finite set called the **action space**;
- $\mathcal{R} \subset \mathbb{R}$ is a finite set of possible **rewards**;
- $p\colon S \times \mathcal{R} \times S \times A \to [0, 1]$ is the **transition-reward probability distribution**.

For $s, s' \in S$, $a \in A$, and $r \in \mathcal{R}$, the quantity $p(s', r \mid s, a)$ is the probability of transitioning to state s' and receiving reward r, given that the current state is s and the agent executes action a. Thus,

$$\sum_{s' \in S} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = 1, \qquad \forall s \in S,\ a \in A.$$

To specify the interaction, we also fix an initial-state distribution $\mu_0 \in \Delta(S)$, where $\Delta(S)$ denotes the set of probability distributions over S. States in the support of μ_0 are the possible initial states. When the task has a termination condition, we denote by $S_{\text{term}} \subseteq S$ the set of *terminal states*.

A *randomized policy* specifies, at each time t, a probability distribution $\pi_t(\cdot \mid s)$ over actions for every state $s \in S$. A policy is *stationary* if this distribution does not explicitly depend on time, so that $\pi_t(\cdot \mid s) = \pi(\cdot \mid s)$ for every t.

The interaction between the agent and the environment generates a sequence of random variables known as a *trajectory*:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots$$

where S_0 is sampled from μ_0, S_t denotes the state at time t, A_t the action sampled from $\pi_t(\cdot \mid S_t)$, R_{t+1} the reward received after executing A_t, and S_{t+1} the resulting state. The variables S_{t+1} and R_{t+1} are jointly distributed according to $p(\cdot, \cdot \mid S_t, A_t)$.

A trajectory starting from a possible initial state and ending when a terminal state is first reached is called an *episode*. A policy is called *proper* if, from every possible initial state, it generates a finite episode with probability one. We will refer to a task as *episodic* if it admits at least one proper policy.

A task has a *finite horizon* H if the interaction is terminated after at most H decision steps, independently of the policy. A task with no terminal states, in which the agent–environment interaction may proceed indefinitely, is called a *continuing task*.

The objective of the RL agent is to find a policy that maximizes the expected *return*, the cumulative reward received from time step t onward. The *discounted return* from time t is defined as

$$J_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

where $\gamma \in [0, 1]$ is the discount factor. In tasks that may continue indefinitely, one generally takes $\gamma < 1$ so that the return is well defined for bounded rewards. For finite-horizon tasks, only finitely many rewards are accumulated, and therefore $\gamma = 1$ is also allowed. The objective is typically to maximize $\mathbb{E}_\pi[J_0]$.

Wagner's Framework: Transition, Horizon, and Reward.

In the context of Wagner's framework [5], given a family $\mathcal{G}$ of graphs, we assume the targeted conjecture to be written in one of the following normal forms:

$$f(G) \leq 0 \quad \text{or} \quad f(G) < 0 \qquad \forall G \in \mathcal{G}.$$

Without loss of generality, let us consider the first case. In Wagner's framework, the environment dynamics are *deterministic*: once the current state s and the action a are fixed, the next state s' is uniquely determined. Any randomness may therefore come from the policy rather than from the environment. The state-transition component can thus be represented by a deterministic transition function $\tau \colon S \times A \to S$, such that $S_{t+1} = \tau(S_t, A_t)$.

The initial state is fixed, so the initial-state distribution μ_0 is concentrated on a single state. Moreover, Wagner's original build-your-graph game is episodic and has a fixed finite horizon H: after H decisions, a final graph G_H has been constructed. Finally, as anticipated, the reward mechanism is determined by the function f. The reward is sparse: all intermediate rewards are zero, while the final reward is the score of the graph constructed by the agent:

$$R_{t+1} = \begin{cases} 0 & \text{if } t < H - 1, \\ f(G_H) & \text{if } t = H - 1. \end{cases}$$

To optimize the policy, Wagner uses the *deep cross-entropy method*. At each training iteration, a batch of episodes is generated using the current policy and ranked

according to the final reward. A top-performing fraction of the episodes is then retained, and the neural network representing the policy is trained to increase the probability of the actions taken in these selected episodes. Repeating this procedure progressively biases the policy toward higher-reward trajectories.
Since the task has finite horizon, we may take $\gamma = 1$. The aim of the RL agent is therefore to find a sequence of actions $(A_0, A_1, \dots, A_{H-1})$ that maximizes the return from the initial state:

$$J_0 = \sum_{t=1}^{H} R_t = R_H = f(G_H).$$

A counterexample is found if the sequence leads to a graph G_H such that $f(G_H) > 0$.

Remark. The description above corresponds to Wagner's original sequential build-your-graph environment, but several components of the framework can be modified. For instance, an "edge-flipping" environment may allow the agent to add or remove an arbitrary edge at each time step. An incremental reward $f(G_t) - f(G_{t-1})$ could replace the sparse reward, just as stochastic environment dynamics could replace the deterministic ones. Analogous variations can be applied to the horizon, the policy optimization component, or the underlying neural network. In Section 6 we describe our experiment, showing how the best-fitting policy network can be chosen for a given conjecture.

4 Neural Networks

Standard notation for feedforward neural networks [19, Ch. 6] was adapted to our purposes.

Definition (Artificial Neuron)
Given an input vector $x \in \mathbb{R}^n$, an *artificial neuron* is a function $f\colon \mathbb{R}^n \to \mathbb{R}$ defined by

$$f(x) = g\left(\sum_{i=1}^{n} w_i x_i + b\right) = g\left(w^T x + b\right),$$

where the weight vector $w = [w_1, w_2, \dots, w_n]^T \in \mathbb{R}^n$ and the bias $b \in \mathbb{R}$ are parameters, while $g\colon \mathbb{R} \to \mathbb{R}$ is a fixed *activation function*, typically nonlinear.

For $w \neq \mathbf{0}$, the equation $w^T x + b = 0$ defines an affine hyperplane in $\mathbb{R}^n$, which may act as a decision boundary in classification settings. The activation function introduces nonlinearity: without nonlinear activations, a composition of affine layers would still define an affine map; see [20]. Common activation functions include the sigmoid $\frac{1}{1+e^{-x}}$, the hyperbolic tangent $\tanh(x)$, and the Rectified Linear Unit (ReLU) $\max(0, x)$.

Artificial neurons operating in parallel on the same input form a neural network layer.

Definition (Neural Network Layer)
Let $k, n_{k-1}, n_k \in \mathbb{N}^+$. A *neural network layer* is a function $F^{(k)}\colon \mathbb{R}^{n_{k-1}} \to \mathbb{R}^{n_k}$ of the form

$$F^{(k)}(x^{(k-1)}) = g^{(k)}\left(W^{(k)} x^{(k-1)} + b^{(k)}\right) = x^{(k)},$$

where $W^{(k)} \in \mathbb{R}^{n_k \times n_{k-1}}$ is the weight matrix, $b^{(k)} \in \mathbb{R}^{n_k}$ is the bias vector, and the activation function $g^{(k)}\colon \mathbb{R}^{n_k} \to \mathbb{R}^{n_k}$ typically acts element-wise. The integers k and n_k are called the *index* and *width* of the layer, respectively.

Finally, stacking neural network layers yields a neural network. Many implementations are possible, but we define below one of the earliest and simplest *feedforward* architectures.

Definition (Multilayer Perceptron)

A *multilayer perceptron* (MLP) of *depth* $d \in \mathbb{N}^+$ is a neural network defining a parametrized class of functions from $\mathbb{R}^{n_0}$ to $\mathbb{R}^{n_d}$, where the integers n_0 and n_d are, respectively, the *input dimension* and *output dimension*.
Its parameters are the weight matrices and bias vectors across the sequence of layers $F^{(1)} \dots F^{(d)}$,

$$\Theta = \{W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}, \dots, W^{(d)}, b^{(d)}\}.$$

For a fixed choice of Θ, the function $f_\Theta : \mathbb{R}^{n_0} \to \mathbb{R}^{n_d}$ the network defines is

$$f_\Theta = F^{(d)} \circ F^{(d-1)} \circ \cdots \circ F^{(1)}.$$

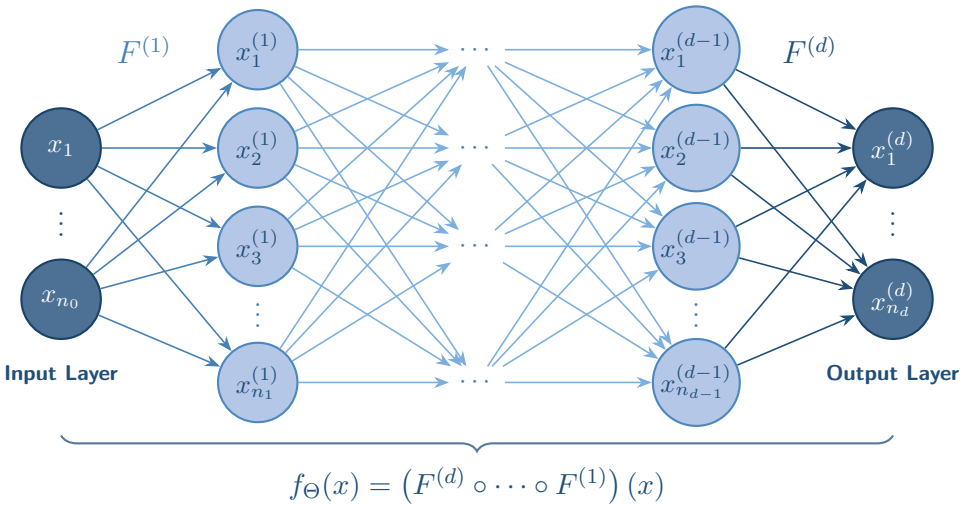


Figure 3: A neural network of depth d. The input $x \in \mathbb{R}^{n_0}$ is sequentially transformed by the layer functions $F^{(k)}$ to produce the final output in $\mathbb{R}^{n_d}$.

Under suitable assumptions, neural networks are universal approximators of continuous functions over compact sets [21]. Given a dataset of p pairs $\{(x_1, y_1), (x_2, y_2), \dots, (x_p, y_p)\}$, where $x_i \in \mathbb{R}^{n_0}$ is an input vector and $y_i \in \mathbb{R}^{n_d}$ its corresponding ground-truth target, *training* an MLP f_Θ consists of choosing the set Θ with the aim of making f_Θ an accurate approximation of the unknown function that produced the sequence of observations. This is done by minimizing a *loss function* measuring the discrepancy between predictions and targets. A common choice for regression tasks is the Mean Squared Error (MSE),

$$\mathcal{L}_{\text{MSE}}(\Theta) = \frac{1}{p} \sum_{i=1}^{p} \|f_\Theta(x_i) - y_i\|^2 .$$

The loss is minimized by iteratively adjusting the network parameters. Identifying all parameters with a single vector, a basic optimization method is *gradient descent*, updating

$$\Theta_{k+1} = \Theta_k - \alpha \nabla \mathcal{L}(\Theta_k),$$

where $\alpha > 0$ is the *learning rate* controlling the size of each update.

The gradient can be efficiently computed through *backpropagation*, which applies the chain rule backwards through the computational graph defined by the composition of the network layers. For a more detailed treatment of these topics, see [19].

Wagner's Framework: Neural Architecture.

To decide whether to include the next edge in the graph, Wagner [5] used an MLP with two input vectors of length $\frac{n(n-1)}{2}$: one encoding the edges selected up to that point, according to a fixed ordering, and the other identifying the edge currently under examination. With the chosen ordering, the first vector corresponds to the flattened strictly upper triangular part of the adjacency matrix $A(G)$.
Although simple and flexible, this representation has a crucial limitation: an MLP does not enforce permutation invariance. This means that different vertex labellings of the same graph generally produce different input vectors and are not guaranteed to be mapped to the same output.

Graph neural networks, also suggested by Wagner [5], address these limitations taking into account the graph topology, by propagating information between adjacent nodes. The graph structure and the node features are separate inputs: even when all nodes are initialized with the same dummy feature, such as $h_v^{(0)} = 1$, the adjacency relation determines how information is exchanged during message passing.

5 Graph Neural Networks

We use the standard neighborhood-aggregation formulation of message passing Graph Neural Networks, with notation adapted to this thesis; see [22, 23].

Definition (Multiset)

A *multiset* is a generalization of a set in which elements may occur more than once, denoted by $\{\{\dots\}\}$. Formally, a finite multiset can be represented by a pair (M, m), where M is the set of distinct elements and $m\colon M \to \mathbb{N}^+$ assigns to each element its multiplicity.

To exploit the graph topology, *Graph Neural Networks* (GNNs) exchange local information through *message passing*. At each layer, nodes update their feature vectors by aggregating information from their neighborhood $\mathcal{N}(v) = \{u \in V : \{u, v\} \in E\}$.

Definition (Message Passing Layer)

Let $G = (V, E)$ be a graph, with an initial feature vector $h_v^{(0)} \in \mathbb{R}^{d_0}$ associated with each node $v \in V$. For each layer $k \geq 1$, a *message passing layer* updates $h_v^{(k-1)} \in \mathbb{R}^{d_{k-1}}$ to $h_v^{(k)} \in \mathbb{R}^{d_k}$ in two steps:

1. **Aggregation:** the features of the neighboring nodes are combined using an operator, such as sum, max, or mean, defined on multisets:
$$m_v^{(k)} = \mathrm{AGGREGATE}^{(k)}\left(\{\{h_u^{(k-1)} : u \in \mathcal{N}(v)\}\}\right).$$
2. **Update:** the aggregated message is combined with the current feature vector of the node:
$$h_v^{(k)} = \mathrm{UPDATE}^{(k)}\left(h_v^{(k-1)}, m_v^{(k)}\right).$$

If the final task requires a graph-level prediction, the node features obtained after the last message passing layer must be combined into a single graph-level representation. This

is done through a permutation-invariant *readout* function:

$$h_G = \text{READOUT}\left(\{\{h_v^{(K)} : v \in V\}\}\right).$$

Common choices for the READOUT function include sum, mean, and max pooling.

Remark. Defining these functions as operating on multisets makes them invariant with respect to index permutation. In an implementation, operations run directly on the feature matrix $H^{(k)} \in \mathbb{R}^{n \times d_k}$, having as rows the transposed feature vectors. Therefore, algorithmic implementations of both AGGREGATE and READOUT must be invariant under any row-wise permutation of $H^{(K)}$ to respect the definition. A relabelling of the graph therefore permutes both the rows of $H^{(k)}$ and the corresponding rows and columns of the adjacency matrix. Moreover, the UPDATE of every node is computed in parallel: every $h_v^{(k)}$ is obtained from the features $H^{(k-1)}$ of the previous layer. Consequently, relabelling the vertices simply relabels the output in the exact same way. In matrix form, for any permutation matrix P,

$$M^{(k)}(PAP^T, PH^{(k-1)}) = P\,M^{(k)}(A, H^{(k-1)}).$$

Thus, a message passing layer is permutation equivariant.

In practice, the UPDATE function is typically parametrized by a neural network. For instance, the current node state and the aggregated message can be concatenated and processed by an MLP:

$$h_v^{(k)} = \text{UPDATE}^{(k)}\left(h_v^{(k-1)}, m_v^{(k)}\right) = f_{\Theta^{(k)}}\left(h_v^{(k-1)} \parallel m_v^{(k)}\right),$$

where $\parallel$ denotes vector concatenation. The same MLP, with parameters $\Theta^{(k)}$, is applied to every node at layer k; different message passing layers may have different parameters.

Definition (Graph Neural Network)

Let $G = (V, E)$ be a graph of order n, with adjacency matrix A, and let $H^{(0)} \in \mathbb{R}^{n \times d_0}$ be its node feature matrix. A message passing graph neural network of depth $K \in \mathbb{N}^+$ applies K message passing layers

$$H^{(k)} = M^{(k)}(A, H^{(k-1)}), \qquad k = 1, \ldots, K.$$

For a graph-level task, the final node representations are combined by a permutation-invariant READOUT function and processed by a final neural network $f_{\Theta_{\text{out}}}$:

$$GNN_\Omega(A, H^{(0)}) = f_{\Theta_{\text{out}}}\left(\text{READOUT}\left(\{\{h_v^{(K)} : v \in V\}\}\right)\right) \in \mathbb{R}^{d_{\text{out}}}.$$

The global parameter set

$$\Omega = \{\Theta^{(1)}, \ldots, \Theta^{(K)}, \Theta_{\text{out}}\}$$

contains the parameters of all message passing layers together with those of the final predictive network. The integers d_0 and d_{out} are respectively called the *input feature dimension* and the *output dimension* of the graph neural network.

This definition refers to models suited for graph-level tasks, such as those of interest in this thesis. In node-level tasks, instead of aggregating node features with the READOUT function, the final MLP is applied independently to each node's updated feature vector to produce a prediction

$$y_v = f_{\Theta_{\text{out}}}\left(h_v^{(K)}\right).$$

Not every GNN is based on message passing, but the most commonly used models are. For this reason, in the following we will refer to Message Passing GNNs simply as GNNs.

Theorem (Invariance and Equivariance of GNNs)**.** *A Message Passing GNN is permutation equivariant at the node level, and its corresponding graph-level output is permutation invariant.*

Proof. The result follows directly from the definitions and the remark above. At the node level, the AGGREGATE function operates on multisets, making the local message $m_v^{(k)}$ independent of the ordering of the neighborhood $\mathcal{N}(v)$. Since the same UPDATE function is applied independently and in parallel to all vertices, relabelling the nodes simply relabels the resulting node representations. Thus, for any permutation matrix P,

$$M^{(k)}(PAP^T, PH^{(k-1)}) = P\, M^{(k)}(A, H^{(k-1)}),$$

so each message passing layer is permutation equivariant.

Finally, the graph-level READOUT function is invariant under permutations of the final node representations. Hence its output is unchanged by a relabelling of the graph, and applying the final $f_{\Theta_{\text{out}}}$ preserves this invariance. □

5.1 Graph Convolutional Network

One of the most popular yet simple GNN architectures is known as *Graph Convolutional Network* (GCN). Here is the formal structure of a single layer [24].

Definition (GCN Layer)
Let $H^{(k)} \in \mathbb{R}^{n \times d_k}$ be the node feature matrix at layer k. A *GCN layer* updates the node features according to

$$H^{(k+1)} = \sigma\left(\widetilde{D}^{-\frac{1}{2}}\widetilde{A}\widetilde{D}^{-\frac{1}{2}}H^{(k)}W^{(k)}\right),$$

where:

- $W^{(k)} \in \mathbb{R}^{d_k \times d_{k+1}}$ is the trainable weight matrix;
- $\widetilde{A} = A + I_n$ is the adjacency matrix of the graph G augmented with self-loops;
- $\widetilde{D} \in \mathbb{R}^{n \times n}$ is the diagonal degree matrix of $\widetilde{A}$, where $\widetilde{D}_{ii} = \sum_j \widetilde{A}_{ij}$;
- σ is an element-wise nonlinear activation function, such as ReLU.

A GCN layer is a degree-weighted version of the message passing scheme. Relative to the unweighted multiset formula above, it additionally uses coefficients determined by the degrees at both endpoints of each edge. The AGGREGATE step is a symmetrically degree-normalized sum over the neighborhood, including the node itself through the added self-loop. The aggregated features are then transformed by the shared weight matrix $W^{(k)}$ and the activation function σ.

The displayed formula omits bias terms; an implementation may add a shared bias before the activation, as in the experiments of Section 6. The symmetric degree normalization controls the contribution of nodes with different degrees and the scale of the propagated features, improving numerical stability when message passing is repeatedly applied across several layers.

Equivalently, if $\widetilde{d}_i = \deg(i) + 1$ is the degree of node i in $\widetilde{A}$, the update rule for a single node feature vector is

$$h_i^{(k+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\widetilde{d}_i \widetilde{d}_j}} (W^{(k)})^T h_j^{(k)}\right).$$

5.2 Graph Isomorphism Problem and 1-WL Test

As anticipated in Section 2, determining whether two given graphs are isomorphic is a difficult computational problem. It is not known whether graph isomorphism can be decided in polynomial time. However, Babai [25] showed that the problem can be solved in quasipolynomial time.

A much simpler polynomial-time procedure is the *1-Weisfeiler–Leman test* (1-WL), also known as *color refinement* [26]. The test does not solve the graph isomorphism problem in general, but can distinguish many pairs of non-isomorphic graphs by iteratively refining vertex colors according to the colors of neighboring vertices.

For an unlabelled graph, all vertices are initially assigned the same color:

$$\lambda_0(v) = c \qquad \text{for every } v \in V.$$

More generally, if initial node labels or features are available, they can be used to determine the initial coloring. Given a coloring λ_k, the next coloring is obtained by assigning to each vertex v the pair consisting of its current color and the multiset of the colors of its neighbors:

$$\lambda_{k+1}(v) = (\lambda_k(v), \{\{\lambda_k(u) : u \in \mathcal{N}(v)\}\}).$$

In practice, each distinct pair is replaced by a new color, with equal pairs receiving the same color. The procedure repeatedly refines the current partition of the vertices into color classes. It stops when no further refinement occurs, that is, when

$$\lambda_{k+1}(u) = \lambda_{k+1}(v) \quad \Longleftrightarrow \quad \lambda_k(u) = \lambda_k(v)$$

for every pair of vertices u, v. The resulting coloring is called the *stable coloring*. Since every non-trivial refinement increases the number of color classes, a graph with n vertices stabilizes after at most $n - 1$ proper refinement steps.

To compare two graphs G_1 and G_2, the refinement can be performed simultaneously on their disjoint union, so that the same colors are assigned according to the same refinement rule. If at some iteration the multisets of colors occurring in the two graphs differ, then G_1 and G_2 cannot be isomorphic. The converse does not hold: there exist non-isomorphic graphs that 1-WL cannot distinguish.

The Weisfeiler–Leman procedure can be generalized to higher dimensions. The k-WL test assigns colors to k-tuples of vertices and iteratively refines them according to suitable neighboring tuples. Increasing k generally improves the distinguishing power of the procedure [26].

Remark. An iteration of 1-WL closely resembles a message passing layer. The current color $\lambda_{k-1}(v)$ plays the role of the node feature, while the multiset

$$\{\{\lambda_{k-1}(u) : u \in \mathcal{N}(v)\}\}$$

contains the information aggregated from the neighborhood. The new color $\lambda_k(v)$ is then an injective encoding of the pair formed by the previous color and this neighborhood multiset. Thus, 1-WL can be viewed as an idealized message passing procedure in which no information is lost during either aggregation or update. This connection will be useful in the next section when comparing the expressive power of different GNN architectures.

5.3 Graph Isomorphism Network

As discussed above, a GNN recursively updates each node representation by aggregating the multiset of features in its neighborhood. Xu et al. [22] studied the expressive power of this class of GNNs and its connection with the 1-WL test.

Intuitively, if two different neighborhood multisets are mapped to the same representation by aggregation or update, information is lost at that step. Later layers may still distinguish the nodes using other neighborhood information, but such collisions can reduce the network's expressive power. Injective aggregation and update functions therefore provide the greatest possible distinguishing power within this framework.

As shown by Xu et al. [22], the 1-WL test gives an upper bound on the expressive power of message passing GNNs.

Lemma. *Let G_1 and G_2 be two graphs with corresponding initial node features. If a message passing GNN maps G_1 and G_2 to different graph representations, then the 1-WL test also distinguishes G_1 and G_2.*

The intuition follows from the previous Remark. At each iteration, 1-WL retains the complete pair consisting of the current color of a node and the multiset of colors of its neighbors. A GNN may instead map different such pairs to the same feature vector whenever its aggregation or update functions are not injective. It can therefore lose distinctions made by 1-WL, but cannot create distinctions between graphs that 1-WL is unable to distinguish. For a formal proof, see [22, Lemma 2].

The natural question is whether a GNN can attain this upper bound. If the aggregation and update functions, together with the graph-level readout, are injective, then it can.

Theorem (Expressive Power of GNNs vs. 1-WL Test)**.** *Let G_1 and G_2 be two graphs, and consider a GNN with K message passing layers. Assume that initial node features correspond one-to-one to the initial 1-WL colors. Suppose that, for every $k \in \{1, \dots, K\}$,* $\mathrm{AGGREGATE}^{(k)}$ *is injective over multisets and* $\mathrm{UPDATE}^{(k)}$ *is injective over its pair of inputs. Suppose also that* READOUT *is injective over multisets.*

If the 1-WL test distinguishes G_1 and G_2 after K iterations, then their graph representations

$$h_G = \mathrm{READOUT}\left(\{\{h_v^{(K)} : v \in V(G)\}\}\right)$$

are different.

Proof. Initialize 1-WL so that its initial colors correspond one-to-one to the initial node features. We prove by induction that, for every k, there exists an injective function g_k such that

$$h_v^{(k)} = g_k(\lambda_k(v)).$$

The statement holds for $k = 0$ by the choice of the initial coloring. Assume that it holds at layer $k-1$. Then

$$h_v^{(k)} = \mathrm{UPDATE}^{(k)}\left(g_{k-1}(\lambda_{k-1}(v)), \mathrm{AGGREGATE}^{(k)}\left(\{\{g_{k-1}(\lambda_{k-1}(u)) : u \in \mathcal{N}(v)\}\}\right)\right).$$

Since g_{k-1}, $\mathrm{AGGREGATE}^{(k)}$, and $\mathrm{UPDATE}^{(k)}$ are injective, the right-hand side is an injective function of

$$(\lambda_{k-1}(v), \{\{\lambda_{k-1}(u) : u \in \mathcal{N}(v)\}\}) = \lambda_k(v).$$

Hence there exists an injective g_k such that $h_v^{(k)} = g_k(\lambda_k(v))$.

If 1-WL distinguishes G_1 and G_2 after K iterations, then

$$\{\{\lambda_K(v) : v \in V(G_1)\}\} \neq \{\{\lambda_K(v) : v \in V(G_2)\}\}.$$

By injectivity of g_K, the corresponding multisets of final node representations are different, and injectivity of READOUT therefore gives different graph representations. □

The theorem shows that injective aggregation and update functions allow a message passing GNN to match the distinguishing power of 1-WL. Xu et al. [22] used this observation to introduce the *Graph Isomorphism Network* (GIN).

For a countable set $\mathcal{X}$, they showed that there exists a function $f : \mathcal{X} \to \mathbb{R}^d$ such that

$$\sum_{x \in X} f(x)$$

uniquely represents every bounded-size multiset X over $\mathcal{X}$. Moreover, for suitable choices of ϵ, the expression

$$(1+\epsilon) f(c) + \sum_{x \in X} f(x)$$

uniquely represents the pair consisting of an element c and a bounded multiset X [22]. This motivates using sum aggregation together with an MLP $f_\Theta^{(k)}$ trained to model f, to obtain an expressive message passing architecture.

Definition (GIN Layer)

Let $h_v^{(k-1)}$ be the node feature vector of node v at layer $k-1$. A *GIN layer* updates the node representation according to

$$h_v^{(k)} = f_\Theta^{(k)} \left(\left(1 + \epsilon^{(k)}\right) h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)} \right),$$

where $\epsilon^{(k)}$ may be fixed or learned.

Under the injectivity assumptions above, GIN can attain the distinguishing power of the 1-WL test [22]. This cannot in general be guaranteed for GCN. Indeed, GCN does not satisfy the injectivity assumptions of the theorem. Its normalized aggregation can map different neighborhoods to the same representation. For example, suppose that all nodes have the same initial feature. In any regular graph (i.e. any graph where every vertex has the same degree), the normalized aggregation

$$\sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\widetilde{d}_i \widetilde{d}_j}} h_j$$

returns the same value independently of the degree. Thus, for instance, C_6 and $K_{3,3}$ produce the same node representations after a GCN layer, although they have degrees 2 and 3, respectively. In contrast, 1-WL distinguishes them after one iteration because the corresponding neighborhood multisets have different cardinalities. Since both graphs have six vertices, this distinction is not recovered by either sum or mean graph-level pooling.

Moreover, the GCN update consists of a linear transformation followed by an activation function. Common choices such as ReLU are not injective, so the update does not in general preserve distinct aggregated representations. Consequently, while GIN is designed to attain the expressive power of 1-WL under the assumptions above, GCN may lose structural information during message passing.

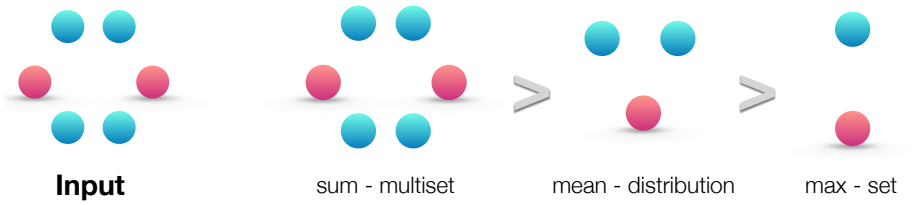


Figure 4: Ranking by expressive power for sum, mean and max aggregators over a multiset. Left panel shows the input multiset, i.e., the network neighborhood to be aggregated. The next three panels illustrate the aspects of the multiset a given aggregator is able to capture: sum, after a suitable injective feature encoding on bounded multisets, can capture the full multiset; mean captures the proportion/distribution of elements of a given type, and the max aggregator ignores multiplicities (reduces the multiset to a simple set). Reproduced from [22, Fig. 2].

6 Experimental Comparison on Spectral Radius

The effectiveness of a reinforcement learning agent in Wagner's framework may depend on the representational capacity of its underlying neural network. Working under this hypothesis, we compared GCN and GIN on a supervised regression task: predicting the adjacency spectral radius $\rho(A(G))$, which for a simple undirected graph coincides with its largest adjacency eigenvalue. This experiment grew out of the collaboration with the CuriosAI research group on Brouwer's conjecture, recently proved by Kothari and Tudose [17].

6.1 Dataset and Experimental Protocol

We used the CuriosAI graph dataset, reported as consisting of 313,597 simple, undirected, pairwise non-isomorphic graphs of order between 2 and 50, generated using an entropy-based criterion favoring comparatively rare invariant values. Its annotations include order, diameter, matching number, clique number, independence number, adjacency spectral radius and Laplacian spectral radius. Each sampled graph was uploaded to the experimental notebook as a triple of features: constant node features $h_v^{(0)} = 1$, its edge list, and its spectral radius. The first two entries were then supplied to the network, while the spectral radius served as a prediction target.

After using `random_state=42` as seed for pseudo-random shuffling, a subset of 50,000 graphs was selected and split into 40,000 training, 5,000 validation and 5,000 test graphs. This data split was used across all models and random seeds. The main comparison was conducted under the same architectural conditions: three message passing layers, global mean pooling, and hidden width of 64. This choice generated an imbalance in the number of trainable parameters (GIN had 20,993 while GCN 8,513), for this reason we conducted further experiments to check the performance of parameter-matched models. The implemented GIN used $\epsilon = 0$ in every layer, following the `GINConv` defaults in PyTorch Geometric 2.6.1 [27], and GCN used the default self-loops, symmetric normalization and trainable biases. All runs used Adam optimizer, learning rate 10^{-3}, batch size 32 and at most 500 epochs. Early stopping was triggered by 50 consecutive epochs without a strict improvement in validation MSE. The checkpoint with the lowest validation MSE was then reloaded and evaluated once on the test set.

For consistency, two other experiments were made using 43 and 44 as seeds for initialization and minibatch shuffling. Reported uncertainties are standard deviations over these three runs, with different random seeds but the same training, validation, and test split.

6.2 Results, Interpretation and Limitations

As a reference, a constant predictor was fitted using the mean training target, 13.3180, and obtained test MSE 113.7849. To examine the effect of parameter count, we additionally trained GCN with hidden width 101, enough to reach 20,908 parameters (only 85 fewer than GIN). The split, seeds and training protocol remained unchanged.

Table 1: Test MSE on the same 5,000 test graphs, mean ± sample standard deviation over three seeds. The constant baseline has no seed-dependent uncertainty.

Model	Width	Parameters	Test MSE
Constant baseline	–	–	113.7849
GCN	64	8,513	11.4257 ± 0.8455
GCN, parameter-matched	101	20,908	12.7500 ± 0.3478
GIN, $\epsilon = 0$	64	20,993	0.0089 ± 0.0026

GIN obtained a test MSE more than three orders of magnitude below either GCN variant (Table 1 and Figure 5), and increasing the GCN parameter count did not close the gap (under this protocol). All three models improved substantially on the constant baseline. More detailed results can be found in the accompanying notebook.

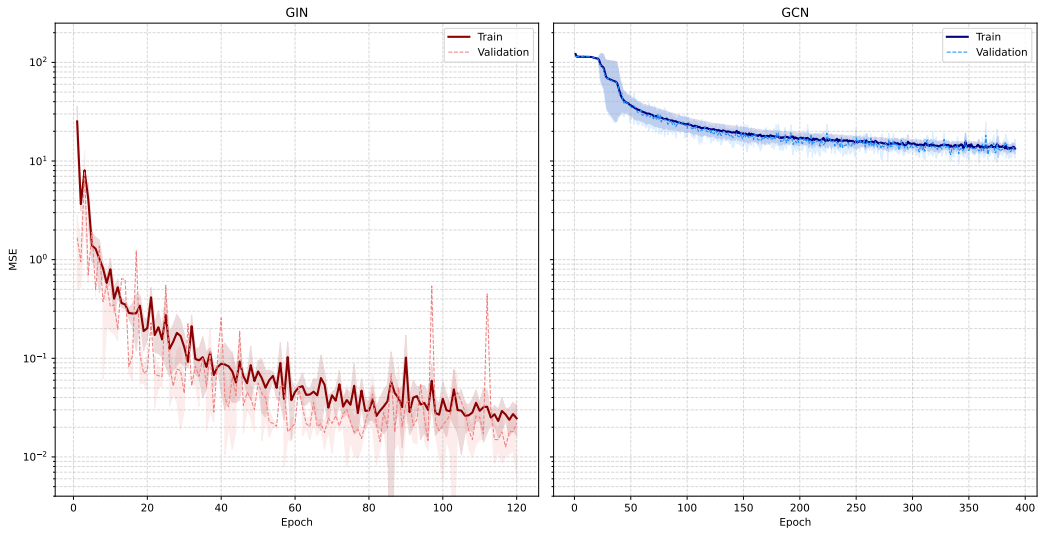


Figure 5: Training and validation MSE on a common logarithmic scale. The curves represent the mean over the three runs, while the shaded regions show ± one sample standard deviation. Because early stopping produced histories of different lengths, the curves were truncated separately for each architecture to the shortest run so that every plotted mean and standard deviation is based on all three seeds.

The experiments showed a substantial difference in performance between GIN and GCN, and the parameter-count control checked that matching the number of trainable parameters is insufficient to even out this disparity. All of this is consistent with the structural properties discussed in Section 5.3.

Nevertheless, these results are not enough to establish that expressivity alone caused the measured error gap: only one choice of hyperparameters, one split, and three seeds were tested. Moreover, the implemented GIN uses the default non-trained $\epsilon = 0$ and a non-injective mean readout function to avoid numerical instability: it is not guaranteed to attain full 1-WL expressivity under these conditions.

7 Conclusion and Future Work

This thesis studied the expressive power of graph neural networks in the context of Wagner's reinforcement learning framework for finding counterexamples to graph-theoretic conjectures. Under the hypothesis that an RL agent's efficiency in exploring graph search spaces may depend on the representational power of its underlying neural network, we theoretically and empirically compared the performances of Graph Convolutional Networks and Graph Isomorphism Networks. We discussed the connection between message passing and the 1-WL test, and the injectivity conditions under which a GNN can match its distinguishing power.

On the supervised task of predicting adjacency spectral radius, GIN achieved substantially lower test MSE than GCN, and the difference persisted when GCN was widened to have approximately the same number of trainable parameters. Good empirical accuracy does not require satisfying every assumption of the 1-WL expressivity theorem: to manage instability, the best-performing implementation used a non-injective mean readout and therefore does not satisfy all the assumptions of the theorem.

Our simple yet accurate model can now be chosen as a benchmark for a policy network to tackle problems involving the spectral radius, or adjusted for different target features and further experiments. The most natural step would be to compare GCN and GIN inside an actual build-your-graph environment, to determine whether the advantage observed in spectral-radius prediction translates into a more effective policy for graph construction. This would clarify whether supervised prediction accuracy is a reliable criterion for selecting the architecture of an RL agent. Further comparisons between the two models could examine alternative pooling operations, trainable ϵ, additional node features, and performance across different graph orders and families.

Before concluding, we invite the readers back to a broader reflection on this pioneering framework. The main contribution of Wagner's approach is not to replace proof, but to direct the search toward promising regions of a combinatorial space that would be too large to examine exhaustively. A counterexample found by the agent can be verified independently and becomes a rigorous result; failure to find one, on the other hand, provides no proof that the conjecture is true. The method should therefore be understood as a new discovery tool whose output must ultimately be treated "the old way", formally. Even without finding a counterexample, the search may identify high-scoring graphs, recurring structural patterns, or possible extremal families, thereby suggesting how a conjecture might be refined or tackled theoretically.

Within this framework, GNNs provide permutation-aware representations of graph-structured data, taking adjacency relations into account while respecting vertex relabelling. Still, a delicate tradeoff must be made when choosing the right model. One that misses relevant distinctions during message passing may limit the agent before it has the opportunity to explore effectively. Conversely, greater expressive power alone does not imply better search performance, since numerical stability, computational cost, and robustness among different graph sizes must also be considered. A promising GNN in a reinforcement learning environment becomes the core tool for counterexample search: the network determines the structural information available to score possible actions, while rewards from the environment guide the agent in the search. The mathematical value of this approach lies ultimately in its ability to produce explicit objects and interpretable evidence from which new conjectures, constructions, or proofs may emerge.

References

[1] K. Appel and W. Haken. "Every planar map is four colorable". In: *Bulletin of the American Mathematical Society* 82 (1976), pp. 711–712.

[2] T. C. Hales. "A proof of the Kepler conjecture". In: *Annals of Mathematics* 162 (2005), pp. 1065–1185.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, et al. "Human-level control through deep reinforcement learning". In: *Nature* 518 (2015), pp. 529–533.

[4] D. Silver, J. Schrittwieser, K. Simonyan, et al. "Mastering the game of Go without human knowledge". In: *Nature* 550 (2017), pp. 354–359.

[5] A. Z. Wagner. *Constructions in combinatorics via neural networks*. 2021. arXiv: 2104.14516 [math.CO].

[6] R. Rubinstein. "The Cross-Entropy Method for Combinatorial and Continuous Optimization". In: *Methodology and Computing in Applied Probability* 1 (1999), pp. 127–190.

[7] A. Davies, P. Veličković, L. Buesing, et al. "Advancing mathematics by guiding human intuition with AI". In: *Nature* 600 (2021), pp. 70–74.

[8] A. Mehrabian, A. Anand, H. Kim, et al. "Finding Increasingly Large Extremal Graphs with AlphaZero and Tabu Search". In: *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 2024, pp. 6985–6993.

[9] Đ. Stevanović, I. Damnjanović, and D. Stevanović. *Finding counterexamples for a conjecture of Akbari, Alazemi and Anđelić*. 2021. arXiv: 2111.15303 [math.CO].

[10] M. Ghebleh, S. Al-Yakoob, A. Kanso, et al. "Reinforcement learning for graph theory, I: Reimplementation of Wagner's approach". In: *Discrete Applied Mathematics* 380 (2026), pp. 468–479.

[11] M. Ghebleh, S. Al-Yakoob, A. Kanso, et al. "Reinforcement learning for graph theory, II. Small Ramsey numbers". In: *The Art of Discrete and Applied Mathematics* 8.1 (2025).

[12] M. Ghebleh, S. Al-Yakoob, A. Kanso, et al. "Graphs having two main eigenvalues and arbitrarily many distinct vertex degrees". In: *Applied Mathematics and Computation* 495, 129311 (2025).

[13] A. Bouffard and J. Breen. *Reinforcement learning for graph theory, Parallelizing Wagner's approach*. 2025. arXiv: 2509.01607 [math.CO].

[14] I. Damnjanović, U. Milivojević, I. Đorđević, et al. *RLGT: A reinforcement learning framework for extremal graph theory*. 2026. arXiv: 2602.17276 [cs.LG].

[15] F. Angileri, G. Lombardi, A. Fois, et al. "A Systematization of the Wagner Framework: Graph Theory Conjectures and Reinforcement Learning". In: *Discovery Science*. Ed. by D. Pedreschi, A. Monreale, R. Guidotti, et al. Vol. 15243. Lecture Notes in Computer Science. Springer, 2025, pp. 325–338.

[16] F. Angileri, G. Lombardi, A. Fois, et al. "Analyzing RL components for Wagner's framework via Brouwer's conjecture". In: *Machine Learning* 114, 242 (2025).

[17] P. K. Kothari and S. Tudose. *On Brouwer's Laplacian conjecture*. 2026. arXiv: 2606.12197 [math.CO].

[18] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. The MIT Press, 2018.

[19] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. The MIT Press, 2016. Chap. 6.

[20] C. Olah. *Neural Networks, Manifolds, and Topology*. 2014. URL: https://colah.github.io/posts/2014-03-NN-Manifolds-Topology/ (visited on 09/21/2026).

[21] K. Hornik. "Approximation Capabilities of Multilayer Feedforward Networks". In: *Neural Networks* 4.2 (1991), pp. 251–257.

[22] K. Xu, W. Hu, J. Leskovec, et al. "How Powerful are Graph Neural Networks?" In: *International Conference on Learning Representations*. 2019. arXiv: 1810.00826.

[23] J. Gilmer, S. S. Schoenholz, P. F. Riley, et al. "Neural Message Passing for Quantum Chemistry". In: *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017*. Ed. by D. Precup and Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017, pp. 1263–1272. URL: http://proceedings.mlr.press/v70/gilmer17a.html.

[24] T. N. Kipf and M. Welling. "Semi-Supervised Classification with Graph Convolutional Networks". In: *International Conference on Learning Representations*. 2017. arXiv: 1609.02907.

[25] L. Babai. *Graph Isomorphism in Quasipolynomial Time*. Revised version. 2016. arXiv: 1512.03547.

[26] C. Morris, M. Ritzert, M. Fey, et al. "Weisfeiler and Leman Go Neural: Higher-Order Graph Neural Networks". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2019, pp. 4602–4609. arXiv: 1810.02244.

[27] PyTorch Geometric team. *GINConv: PyTorch Geometric 2.6.1 documentation*. URL: https://pytorch-geometric.readthedocs.io/en/2.6.1/generated/torch_geometric.nn.conv.GINConv.html (visited on 09/21/2026).